

halfedge：稀疏线性代数模块设计文档

src/linalg.rs

halfedge 库

2026-07-06

目录

1	模块定位	1
2	数据结构：SparseSystem	2
2.1	对称累加语义	2
2.2	索引越界守卫	3
2.3	从 TriMat 到 CsMat	3
3	共轭梯度法	3
3.1	算法公式	3
3.2	流程图	4
3.3	退化守卫	4
4	对角正则化	4
5	三维向量工具：vec3 子模块	5
5.1	类型与函数清单	5
5.2	设计要点	5
5.3	与既有切片函数的关系	6
5.4	消费方一览	6
6	退化守卫汇总	6
7	使用示例	6
8	测试覆盖	7

1 模块定位

linalg.rs 是 halfedge 库的**稀疏线性代数工具**模块。在 sprs 的 CsMat 稀疏矩阵之上提供四层能力：

- SparseSystem：从网格拉普拉斯矩阵的三元组构建稀疏对称系统；

- `conjugate_gradient`: 共轭梯度法求解 $Ax = b$ (A 对称正定);
- 辅助工具: 正则化对角偏移 `regularize_diagonal`、残差范数计算;
- `vec3` 子模块: 全库共用的 `[f64; 3]` 三维向量原语, 消除跨文件重复定义。

使用场景:

- 参数化: 求解调和 / LSCM / Tutte 线性系统;
- 测地线: Heat Method 中的两次 Poisson 求解;
- 共形映射: 离散调和映射;
- 几何 / 拓扑 / 布尔 / 验证等模块: 统一调用 `vec3` 完成点-向量运算。

类别	API	语义
构建器	<code>SparseSystem::new(dim)</code>	新建 $dim \times dim$ 空稀疏系统
	<code>SparseSystem::add(i, j, val)</code>	对称累加 $A_{ij} += val$ (同时写 A_{ji})
构建器	<code>SparseSystem::add_diag(i, val)</code>	仅累加对角元 $A_{ii} += val$
	<code>SparseSystem::finish(self)</code>	转为 CSR 格式 <code>CsMat</code>
	<code>SparseSystem::dim(&self)</code>	返回矩阵维度
求解器	<code>conjugate_gradient(a, b, max_iter, tol)</code>	求解 $Ax = b$; 未收敛返回 <code>None</code>
辅助	<code>regularize_diagonal(a, lambda)</code>	对角正则化 $A \leftarrow A + \lambda I$
	<code>vec3::{sub,add,scale,dot,cross,length,normailze,...}</code>	<code>[f64; 3]</code> 向量原语

2 数据结构: SparseSystem

`SparseSystem` 是一个正在构建中的稀疏对称线性系统。内部用 `sprs::TriMat<f64>` (三元组格式) 收集矩阵元素, 调用 `finish` 后转为 CSR 格式的 `CsMat`, 可直接用于共轭梯度求解。

```

1 pub struct SparseSystem {
2     tri: TriMat<f64>,
3     dim: usize,
4 }
```

2.1 对称累加语义

`add(i, j, val)` 实现对称累加: 当 $i \neq j$ 时同时写入 A_{ij} 与 A_{ji} , 保证矩阵对称; 当 $i = j$ 时只写一次对角元:

$$A_{ij} += val, \quad A_{ji} += val \quad (i \neq j).$$

`add_diag(i, val)` 仅累加对角元 $A_{ii} += val$, 适用于拉普拉斯矩阵中顶点 `degree` 项的填充。

2.2 索引越界守卫

`add` 与 `add_diag` 在写入前检查 $i, j < \text{dim}$, 越界索引静默丢弃 (不 `panic`)。这是面向网格拓扑场景的容错策略: 遍历半边 / 顶点时偶有 `VertexId` 被删除或越界, 应跳过而非中断构建。

2.3 从 TriMat 到 CsMat

`finish(self)` 调用 `TriMat::to_csr()` 完成一次性压缩: 合并重复三元组、按行排序、丢弃显式零元。此后矩阵即不可变, 仅供 `conjugate_gradient` 读取。

3 共轭梯度法

`conjugate_gradient(a, b, max_iter, tol)` 求解对称正定 (SPD) 系统 $Ax = b$ 。算法复杂度 $O(\text{iter} \cdot \text{nnz})$, 远优于直接求逆。

3.1 算法公式

初始化 (取 $x_0 = 0$):

$$r_0 = b - Ax_0 = b, \quad p_0 = r_0, \quad \rho_0 = r_0 \cdot r_0.$$

迭代 ($k = 0, 1, \dots, \text{max_iter} - 1$):

$$\alpha_k = \frac{r_k \cdot r_k}{p_k \cdot Ap_k}, \tag{1}$$

$$x_{k+1} = x_k + \alpha_k p_k, \tag{2}$$

$$r_{k+1} = r_k - \alpha_k Ap_k, \tag{3}$$

$$\beta_k = \frac{r_{k+1} \cdot r_{k+1}}{r_k \cdot r_k}, \tag{4}$$

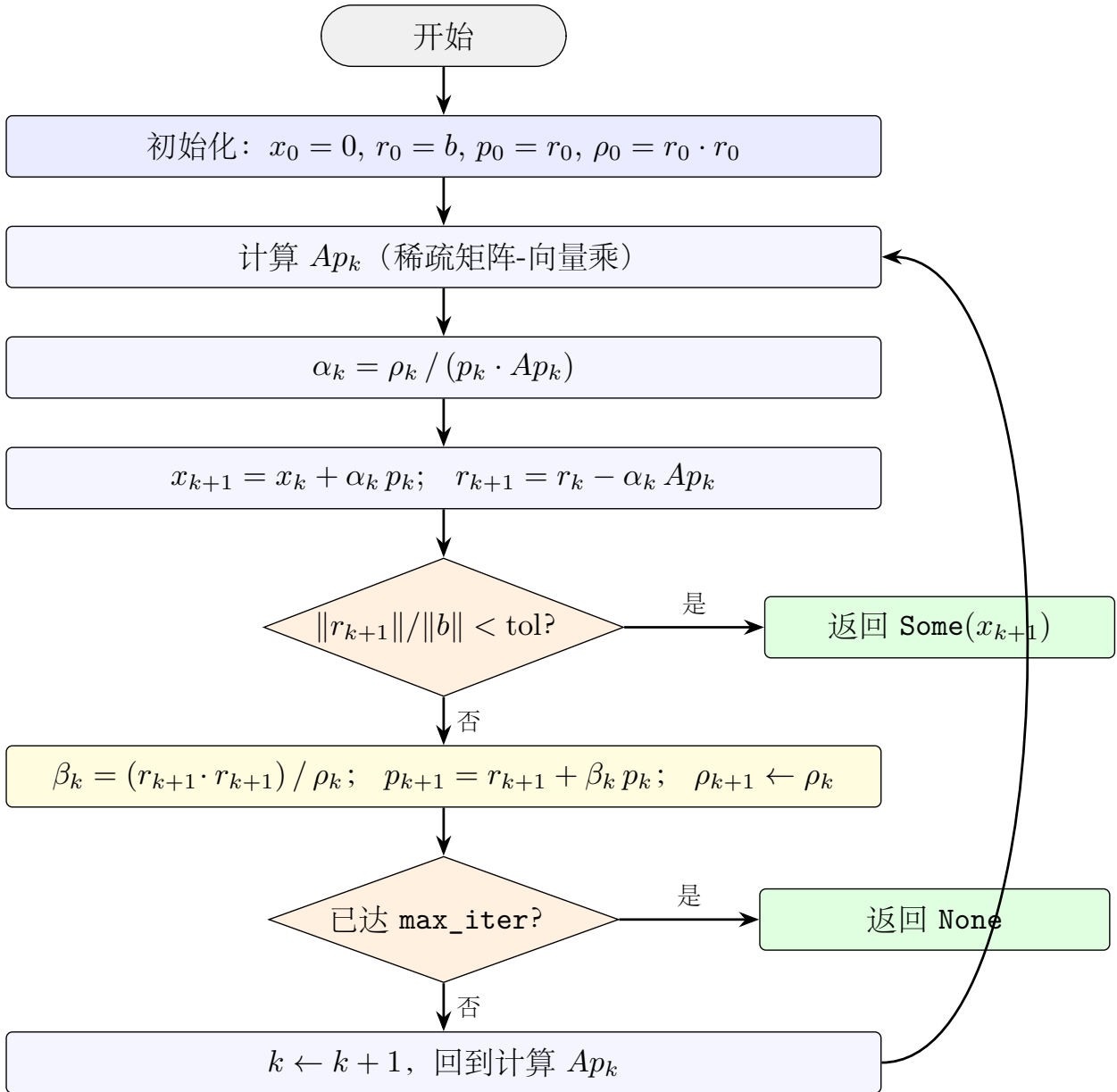
$$p_{k+1} = r_{k+1} + \beta_k p_k. \tag{5}$$

收敛判据 (相对残差):

$$\frac{\|r_k\|_2}{\|b\|_2} < \text{tol} \quad \Rightarrow \quad \text{返回 Some}(x_k).$$

未收敛: 若达到 `max_iter` 仍未满足判据, 返回 `None`, 由调用方决定是否增大迭代数或正则化。

3.2 流程图



3.3 退化守卫

- $n = 0$ (空系统) 直接返回 `Some(Vec::new())`;
- $\|b\| < 10^{-30}$ (右端近似零) 返回零解 `vec![0.0; n]`, 避免除零;
- 维度不匹配 ($n \neq \mathbf{b}.\text{len}()$) 返回 `None` (容错处理, 不 panic)。

4 对角正则化

`regularize_diagonal(a, lambda)` 对稀疏 SPD 矩阵做对角偏移:

$$A_{\text{reg}} = A + \lambda I.$$

用途：处理拉普拉斯矩阵的**零空间**。图拉普拉斯 L 半正定，常零空间维数 ≥ 1 （对应常数向量），CG 不收敛。加上 λI 后 $L + \lambda I$ 严格正定，CG 可收敛；典型 $\lambda \in [10^{-6}, 10^{-3}]$ 。

实现细节：遍历对角索引 $i \in [0, n)$ ，通过 `CsMat::get_mut(i, i)` 就地累加。对拉普拉斯矩阵对角线一定非零（顶点 degree），故无需处理 (i, i) 显式零元的情况。

5 三维向量工具：vec3 子模块

`vec3` 是 `linalg.rs` 内部的 `pub(crate) mod`，为全库提供统一的 `[f64; 3]` 三维向量原语。**动机：**在引入 `vec3` 之前，`geometry.rs`、`geodesics.rs`、`deformation.rs`、`boolean.rs`、`validate.rs`、`test_util.rs` 等多个模块各自重复定义了 `Vec3` 类型别名与 `sub`、`add`、`scale`、`dot`、`cross`、`length`、`normalize` 等函数，导致实现分叉、维护负担增加。统一后所有模块直接 `use crate::linalg::vec3::{...}`；引用，单一来源真理。

5.1 类型与函数清单

类别	API	语义
类型	<code>type Vec3 = [f64; 3]</code>	三维向量类型别名（Copy）
线性运算	<code>sub(a, b) -> Vec3</code>	逐分量相减 $a - b$
	<code>add(a, b) -> Vec3</code>	逐分量相加 $a + b$
	<code>scale(a, s) -> Vec3</code>	标量乘 $s \cdot a$
内/外积	<code>dot(a, b) -> f64</code>	内积 $a \cdot b$
	<code>cross(a, b) -> Vec3</code>	外积 $a \times b$
长度/方向	<code>length(a) -> f64</code>	欧氏范数 $\ a\ _2$
	<code>normalize(a) -> Vec3</code>	单位化； $\ a\ < 10^{-12}$ 时原样返回
	<code>angle_between(u, v) -> f64</code>	夹角（弧度），退化返回 0
三角形	<code>triangle_area(a, b, c) -> f64</code>	$\frac{1}{2} \ (b - a) \times (c - a)\ $
	<code>triangle_normal(a, b, c) -> Vec3</code>	<code>normalize((b - a) × (c - a))</code>

5.2 设计要点

- **传值而非传引用：**因为 `[f64; 3]` 是 Copy 类型，所有函数签名采用 `(a: Vec3, b: Vec3)` 而非 `(&Vec3, &Vec3)`，调用更简洁、避免不必要的借用。
- **统一退化阈值：**`normalize` 与 `angle_between` 使用 10^{-12} 作为长度退化阈值，与 `geometry.rs` 中 `face_normal` 的判定一致。
- **可见性：**`pub(crate)`，仅库内可见，不暴露到公开 API。
- **内联：**所有函数标注 `#[inline]`，编译器可跨模块内联消除调用开销。

5.3 与既有切片函数的关系

`linalg.rs` 另有两个面向 `&[f64]` 切片的辅助函数:

- `pub(crate) fn dot(a: &[f64], b: &[f64]) -> f64`: 任意长度向量内积;
- `pub(crate) fn norm2(v: &[f64]) -> f64`: 任意长度向量 2-范数。

二者用于 `conjugate_gradient` 内部残差计算, 与 `vec3::dot` (定长三维) 语义不同、各司其职: 切片版本服务 CG 求解器, `vec3` 版本服务几何运算。原本二者均为私有 `fn`, 重构后 `direction_field.rs` 需要复用 `norm2`, 故提升为 `pub(crate)`。

5.4 消费方一览

下表列出重构后所有从 `vec3` 引入原语的模块及其用法:

消费模块	引入方式
<code>geometry.rs</code>	<code>use vec3::{Vec3, sub, add, scale, dot, cross, length, normalize, angle_between}</code>
<code>geodesics.rs</code>	<code>use vec3::{self, Vec3}</code> , 调用 <code>vec3::sub(...)</code> 等
<code>deformation.rs</code>	<code>use vec3::{Vec3, sub, add, scale}</code>
<code>boolean.rs</code>	<code>use vec3</code> , 调用 <code>vec3::Vec3</code> 、 <code>vec3::sub(...)</code> 等
<code>validate.rs</code>	<code>use vec3</code> , 仅调用 <code>vec3::triangle_area</code>
<code>test_util.rs</code>	<code>use vec3</code> , 仅调用 <code>vec3::normalize</code>
<code>direction_field.rs</code>	<code>use norm2</code> (切片版本)

6 退化守卫汇总

API	退化场景	处理
<code>SparseSystem::add</code>	$i, j \geq \dim$ (越界)	静默丢弃, 不 panic
<code>SparseSystem::add_diag</code>	$i \geq \dim$ (越界)	静默丢弃, 不 panic
<code>conjugate_gradient</code>	$n = 0$ (空系统)	返回 <code>Some(Vec::new())</code>
<code>conjugate_gradient</code>	$\ b\ < 10^{-30}$	返回零解 <code>vec![0.0; n]</code>
<code>conjugate_gradient</code>	A 与 b 维度不匹配	返回 <code>None</code> (容错, 不 panic)
<code>conjugate_gradient</code>	<code>max_iter</code> 未收敛	返回 <code>None</code>
<code>conjugate_gradient</code>	A 半正定 (非 SPD)	调用方需先 <code>regularize_diagonal</code> 正则化

7 使用示例

```
1 use halfedge::linalg::{SparseSystem, conjugate_gradient, regularize_diagonal};
2
3 // 1.
4 let mut sys = SparseSystem::new(3);
5 sys.add(0, 0, 2.0); //
```

```

6 sys.add(1, 1, 2.0);
7 sys.add(2, 2, 2.0);
8 sys.add(0, 1, -1.0); // (1,0)
9 sys.add(1, 2, -1.0); // (2,1)
10 let mut a = sys.finish();
11
12 // 2. lambda * I SPD
13 regularize_diagonal(&mut a, 1e-4);
14
15 // 3. Ax = b
16 let b = vec![1.0, 0.0, -1.0];
17 match conjugate_gradient(&a, &b, 1000, 1e-10) {
18     Some(x) => println!(" x = {:?}", x),
19     None    => panic!("CG max_iter "),
20 }

```

8 测试覆盖

测试名	验证点
test_cg_identity	$A = I$ 时 CG 一步收敛, $x = b$
test_cg_small_spd	3×3 SPD 系统, 验证 $Ax \approx b$ 至 10^{-10}
test_regularize_diagonal	对角元正确累加 λ , 非对角元不变
test_sparse_system_builder	add 对称写入、add_diag 对角写入正确
cg_dimension_mismatch_returns_none	矩阵 2×2 但 b 长度 3 时返回 None
cg_empty_matrix_returns_empty	0×0 矩阵 + 空 b 返回 Some(Vec::new())
cg_zero_right_hand_side_returns_zero_solution	$b = \vec{0}$ 时返回零解 $\vec{0}$
cg_max_iter_zero_returns_none	max_iter=0 可解系统返回 None (未迭代)
cg_semidefinite_returns_none	$A = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$ (半正定) $p \cdot Ap = 0$ 触发除零守卫返回 None
cg_known_solution_2x2	$A = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix}$, $b = [1, 2]$, 解 $x = [1/11, 7/11]$ 误差 $< 10^{-8}$
sparse_system_add_out_of_bounds_silently_drops	越界索引静默丢弃, 矩阵不含越界条目
sparse_system_add_diag_accumulates	add_diag 两次累加后值为和

src/linalg.rs 共 12 个单元测试, 全库共 552 个。